SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Advanced Data Structures and Design Engineering

Structures, Unions and Storage Classes (50 Marks)

Name	BAVADARANI S
Register No.	1912260562
Course Code &Name	CSA0209 C PROGRAMMING
Slot	B
Assignment/Task Title	Structures, Unions and Storage Classes
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Task 1: Data Design Review (6 Marks)

Task 1.1: Diagnose the Design (2 Marks)

The original design has eight deficiencies. Each is a design problem rather than a syntax problem.

#	Deficiency	What could go wrong in the health centre
1	Parallel arrays: one patient is spread over name[], age[], phone[], bp[] and visitDate[], tied together only by the index.	If one array is sorted, shifted or deleted from without the others, a patient's name gets paired with another patient's BP or phone number. The wrong patient is called, or the wrong BP history is shown to the doctor.
2	A 10-digit phone number is stored in char phone[100][10].	A string of 10 digits needs 11 bytes (the extra one is the terminating '\0'). strcpy writes that 11th byte into the first byte of phone[i+1], because rows are contiguous. The next patient's number is silently damaged, so reminder calls fail. The text is also never validated (letters, wrong length).
3	Blood pressure such as 120/80 is stored in one float.	A float cannot hold two numbers. 120.80, 120.8 and "120/8" cannot be told apart, so systolic and diastolic values cannot be recovered. The hospital cannot test "systolic >= 140 or diastolic >= 90", so hypertension may be missed. Floats are also inexact in binary (120.8 is not stored exactly).
4	The visit date is stored as text in char visitDate[100][8].	"12/11/2026" needs 11 bytes and even "12/11/26" needs 9, so the date overflows into the next row. Text dates cannot be compared or subtracted (for example, to check a 14-day follow-up). Formats are ambiguous (dd/mm or mm/dd) and invalid dates such as 31/02 are accepted.
5	No limit check on count.	The 101st patient is written beyond the arrays, which is undefined behaviour. It can overwrite neighbouring globals (perhaps count itself), crash the program on a busy day, or corrupt earlier records.
6	Everything is a global variable.	Any function can change the arrays or count by accident. A second register (a separate OPD and antenatal clinic, or a test copy) is impossible. The code cannot be unit tested and it is not thread-safe.
7	Sorting by age is hard.	Five arrays must be swapped in lockstep. Forgetting even one swap silently corrupts the whole register. Adding a field such as gender or blood group means changing every swap and every function. Listing older patients first for triage becomes error-prone.
8	addPatient takes five loose parameters and does no checking. Also, name[100][20] holds only 19 characters.	Arguments can be passed in the wrong order (for example, age and BP) and it still compiles. Impossible values (age -5, BP 0) are accepted. A 25-character name overflows into the next record through unchecked strcpy.

Task 1.2: Repair the Design (3 Marks)

Each patient becomes one structure. The date and the blood pressure are nested structures, the phone buffer has room for the terminator, and the array and its count are bundled in one register structure that is passed by pointer. addPatient() now receives a structure pointer and checks capacity.

patient_register.c (compiled and tested)

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100
#define NAME_LEN     30
#define PHONE_LEN    11      /* 10 digits + '\0' */

struct date { int day, month, year; };
struct bp   { int systolic, diastolic; };

struct patient {
```

```
    char        name[NAME_LEN];
    int         age;
    char        phone[PHONE_LEN];
    struct bp   bp;
    struct date visit;
};

struct patient_register {
    struct patient list[MAX_PATIENTS];
    int count;
};

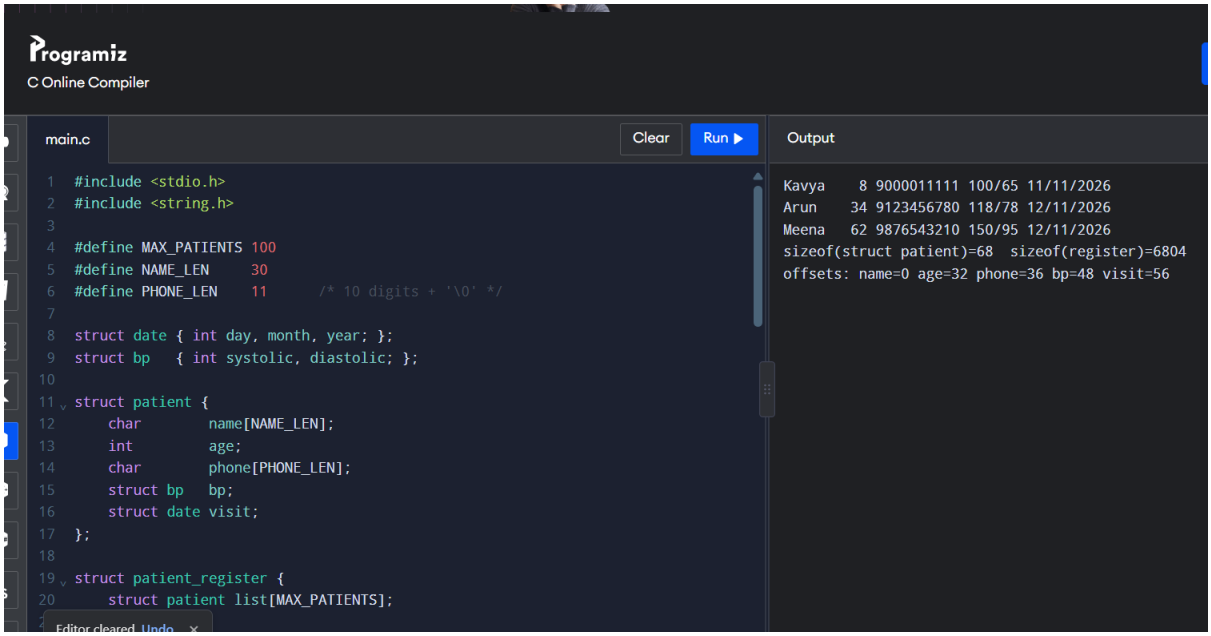
/* returns 1 on success, 0 if the register is full */
int addPatient(struct patient_register *reg, const struct patient *p)
{
    if (reg == NULL || p == NULL || reg->count >= MAX_PATIENTS)
        return 0;
    reg->list[reg->count] = *p;       /* one structure assignment */
    reg->count++;
    return 1;
}

void sortByAge(struct patient_register *reg)
{
    for (int i = 0; i < reg->count - 1; i++) {
        int min = i;
        for (int j = i + 1; j < reg->count; j++)
            if (reg->list[j].age < reg->list[min].age) min = j;
        if (min != i) {
            struct patient t = reg->list[i];
            reg->list[i] = reg->list[min];
            reg->list[min] = t;
        }
    }
}

int main(void)
{
    static struct patient_register reg;          /* count starts at 0 */
    struct patient a = {"Meena", 62, "9876543210", {150, 95}, {12, 11, 2026}};
    struct patient b = {"Arun",  34, "9123456780", {118, 78}, {12, 11, 2026}};
    struct patient c = {"Kavya",  8, "9000011111", {100, 65}, {11, 11, 2026}};
    addPatient(&reg, &a); addPatient(&reg, &b); addPatient(&reg, &c);
    sortByAge(&reg);
    for (int i = 0; i < reg.count; i++)
        printf("%-6s %3d %s %d/%d %02d/%02d/%d\n", reg.list[i].name, reg.list[i].age,
               reg.list[i].phone, reg.list[i].bp.systolic, reg.list[i].bp.diastolic,
               reg.list[i].visit.day, reg.list[i].visit.month, reg.list[i].visit.year);
    printf("sizeof(struct patient)=%zu  sizeof(register)=%zu\n",
           sizeof(struct patient), sizeof(struct patient_register));
    printf("offsets: name=%zu age=%zu phone=%zu bp=%zu visit=%zu\n",
        __builtin_offsetof(struct patient,name), __builtin_offsetof(struct patient,age),
        __builtin_offsetof(struct patient,phone), __builtin_offsetof(struct patient,bp),
        __builtin_offsetof(struct patient,visit));
    return 0;
}
```

Output

```
Kavya    8 9000011111 100/65 11/11/2026
Arun    34 9123456780 118/78 12/11/2026
Meena   62 9876543210 150/95 12/11/2026
sizeof(struct patient)=68  sizeof(register)=6804
offsets: name=0 age=32 phone=36 bp=48 visit=56
```



Changes and why:

- phone[11] holds 10 digits plus '\0'; struct bp keeps systolic and diastolic as separate integers; struct date stores day, month and year as integers, so dates can be compared and subtracted.
- MAX_PATIENTS is checked inside addPatient(), which returns 0 when the register is full. The caller must handle that result.
- No globals. The register is created by the caller and passed as struct patient_register *. Two registers can now coexist.
- reg->list[reg->count] = *p; copies the whole record with one structure assignment. This replaces five strcpy or assignment statements and cannot misalign fields. The input p is const because it is only read.

Task 1.3: Design Defence (1 Mark, 85 words)

Each patient is one self-contained structure, so sorting swaps whole records with a single assignment and the fields can never become mismatched. A pointer passes the entire record, or the whole register, to a function as one argument, avoiding long parameter lists. To extend the design, add a field such as gender to struct patient; existing code keeps working. The nested date and blood-pressure types are reusable and comparable, and the register bundles the array with its count, so capacity checks travel with the data.

Alternative Concept and Complexity Analysis

When are parallel arrays (structure of arrays, SoA) still preferred?

- When most operations scan **one field** across many records (for example, the average age or "count patients older than 60"). The age[] array is contiguous (4-byte stride), whereas in an array of structures the ages are 68 bytes apart, so far fewer cache lines are loaded.
- When the compiler or hardware can vectorise (SIMD or GPU) a single-field loop, and in column-store databases and analytics.
- When each field is stored on its own, for example only one column must be written to disk, or fields are added and dropped independently of each other.
- Padding is avoided: arrays of one type are packed with no gaps between elements.

Sorting n patients by age (selection sort, at most n-1 swaps). Both designs make n(n-1)/2 comparisons, so the time is O(n^2) in both (O(n log n) with merge sort or qsort). They differ in the cost of one swap:

	Array of structures (AoS)	Parallel arrays (SoA)
Elements moved per swap	1 element: the whole structure (68 bytes)	5 elements, one in each array (20+4+10+4+8 = 46 bytes in the original sizes)
Copy work per swap (temp-based)	3 x 68 = 204 bytes, in 3 statements	3 x 46 = 138 bytes, in 15 statements (5 arrays x 3)
Worst-case bytes copied while sorting	up to 204(n-1) bytes	up to 138(n-1) bytes
Comparison phase	Stride of 68 bytes between ages: poorer cache use	Stride of 4 bytes: better cache use
Risk	None (the record stays together)	High: a missed array swap corrupts the data
Space (corrected field sizes)	100 x 68 + 4 = 6804 bytes (measured). 65 useful bytes per record + 3 bytes of padding (2 after name, 1 after phone) = 300 bytes of padding, about 4.4%.	100 x (30+4+11+8+12) + 4 = 6504 bytes with no padding between records

The flawed original uses only 4604 bytes (measured: 2000+400+1000+400+800+4), but only because its fields are too small, which is exactly the overflow problem diagnosed above. For either design, sorting an array of **indices or pointers** (4 to 8 bytes per swap) removes most of the copying cost.

Task 2: From Requirement to Nested Structures (8 Marks)

Task 2.1: Identify the Entities (2 Marks)

Entity	Fields	Nested inside
Parcel (main entity)	id, weight, booking date, sender, receiver, service type, charge	none (it is the top-level structure)
Person (sender / receiver)	name, address	Parcel (used twice: sender and receiver)
Address	city, PIN code	Person
Date	day, month, year	Parcel (booking date)
Service type	NORMAL or EXPRESS (an enumeration)	Parcel

Nesting diagram

```
Parcel
 |-- id
 |-- weight
 |-- booked    : Date    { day, month, year }
 |-- sender    : Person  { name, addr : Address { city, pincode } }
 |-- receiver  : Person  { name, addr : Address { city, pincode } }
 |-- type      : enum service { NORMAL, EXPRESS }
 '-- charge
```

Task 2.2: Define the Structures (2 Marks)

```
#define MAX_PARCELS 100

enum service { NORMAL, EXPRESS };

typedef struct { int day, month, year; } Date;
typedef struct { char city[20]; int pincode; } Address;
typedef struct { char name[30]; Address addr; } Person;
```

```
typedef struct {
    int      id;
    float    weight;
    Date     booked;
    Person   sender;
    Person   receiver;
    enum service type;
    float    charge;
} Parcel;

Parcel bookings[MAX_PARCELS];
int    booking_count = 0;
```

Because Address is defined once and reused inside Person, and Person is used for both sender and receiver, no field is declared twice.

Task 2.3: Design the Operations (3 Marks)

(a) Sort the parcels by weight (selection sort, ascending)

```
SORT_BY_WEIGHT(P[0..n-1])
  for i <- 0 to n-2
      min <- i
      for j <- i+1 to n-1
          if P[j].weight < P[min].weight then min <- j
      if min != i then
          temp   <- P[i]          // whole structure copied
          P[i]   <- P[min]
          P[min] <- temp
```

Time: O(n^2) comparisons and at most n-1 structure swaps.

(b) List all parcels for a given receiver city

```
LIST_BY_CITY(P[0..n-1], city)
  found <- 0
  for i <- 0 to n-1
      if P[i].receiver.addr.city equals city      // strcmp(...) == 0
          print P[i].id, P[i].weight, P[i].receiver.name
          found <- found + 1
  if found = 0 then print "No parcels for ", city
```

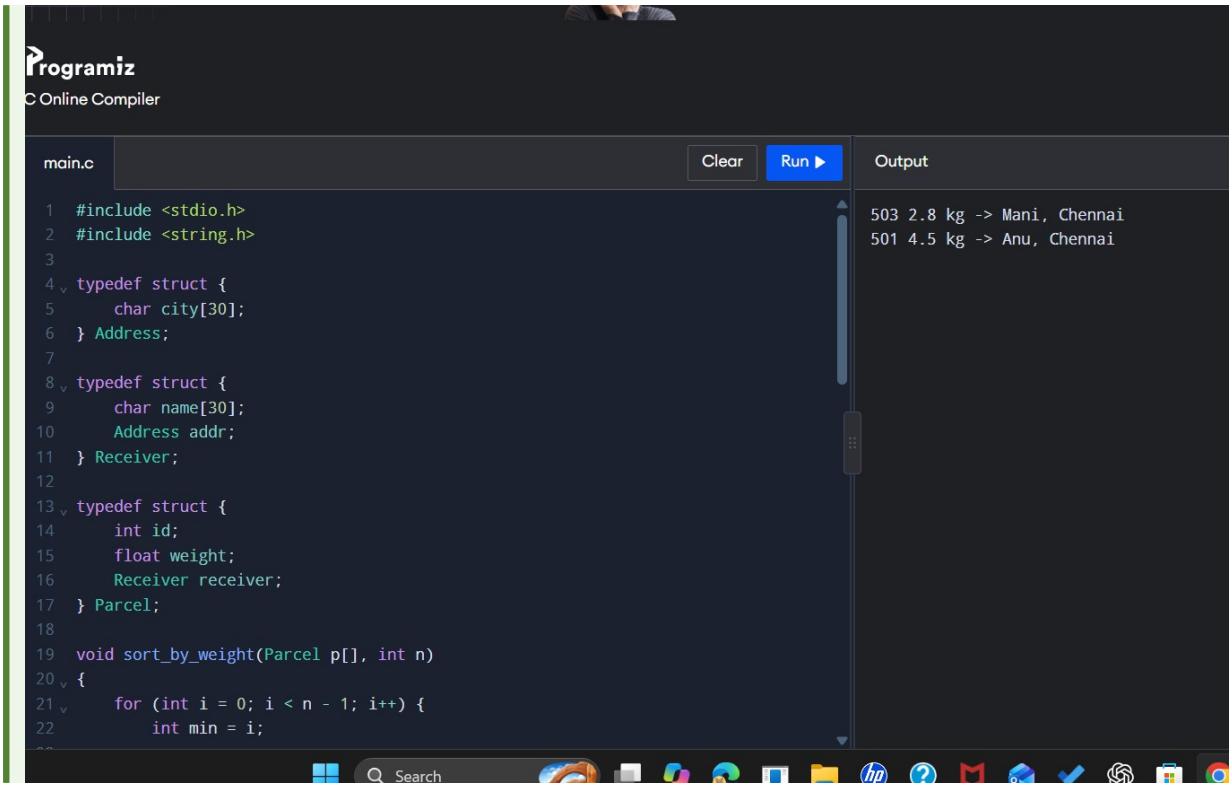
Time: O(n), one linear pass. A working C version of both operations (tested with three sample parcels):

```
void sort_by_weight(Parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) { Parcel t = p[i]; p[i] = p[min]; p[min] = t; }
    }
}

void list_by_city(const Parcel p[], int n, const char *city)
{
    int found = 0;
    for (int i = 0; i < n; i++)
        if (strcmp(p[i].receiver.addr.city, city) == 0) {
            printf("%d %.1f kg -> %s, %s\n", p[i].id, p[i].weight,
                   p[i].receiver.name, p[i].receiver.addr.city);
            found++;
        }
    if (!found) printf("No parcels for %s\n", city);
}
```

Output

```
503 2.8 kg -> Mani, Chennai
501 4.5 kg -> Anu, Chennai
```



Task 2.4: Access Paths (1 Mark)

The PIN code of the receiver of the **third** parcel (index 2, since arrays start at 0):

Method	Expression
(i) Array and dot operator	bookings[2].receiver.addr.pincode
(ii) Structure pointer and arrow operator	Parcel *p = bookings; then (p + 2)->receiver.addr.pincode (equivalent: p[2].receiver.addr.pincode or (&bookings[2])->receiver.addr.pincode)

The arrow is used only once, for the pointer. After that, receiver and addr are structures, not pointers, so the dot operator is used. Both expressions printed 600077 in the test.

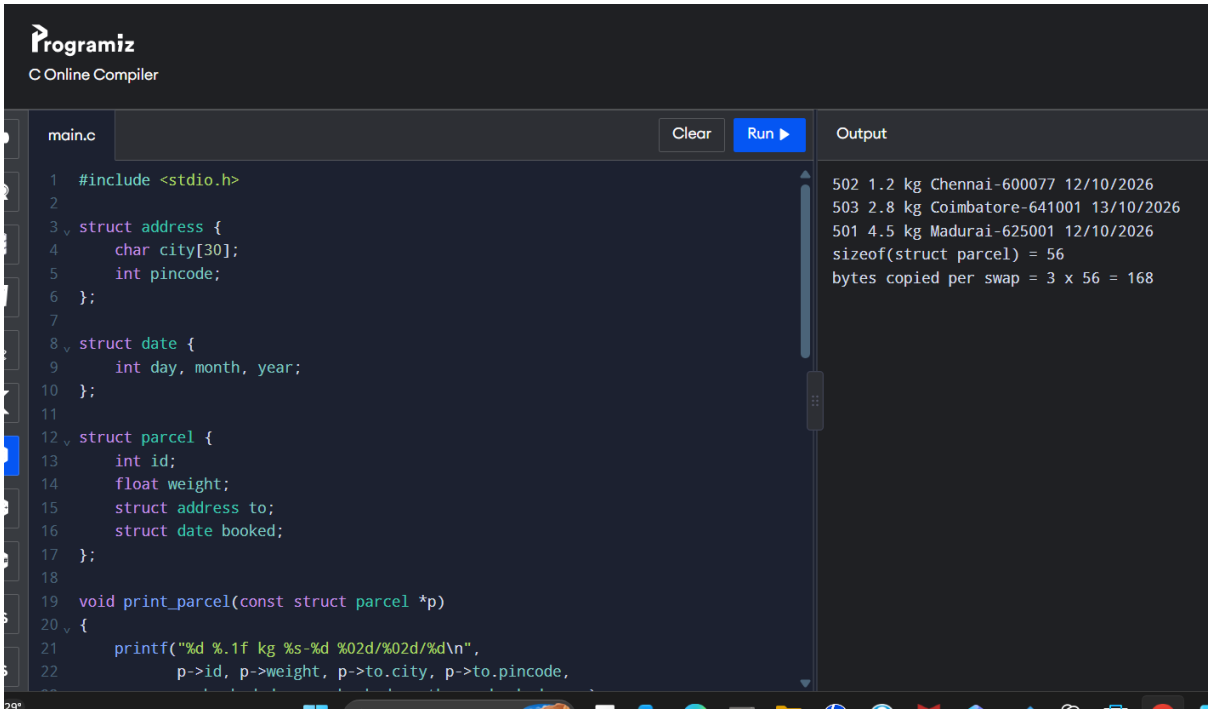
Complete the Code: TODO 1 and TODO 2

```
void print_parcel(const struct parcel *p)
{
   printf("%d %.1f kg %s-%d %02d/%02d/%d\n",
          p->id, p->weight, p->to.city, p->to.pincode,
          p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n)
{
   for (int i = 0; i < n - 1; i++) {
      int min = i;
      for (int j = i + 1; j < n; j++)
         if (p[j].weight < p[min].weight) min = j;
      if (min != i) {
         struct parcel tmp = p[i];
         p[i] = p[min];
         p[min] = tmp;
      }
   }
}
```

Output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
sizeof(struct parcel) = 44, bytes copied per swap = 3 x 44 = 132
```



Bytes copied per swap. sizeof(struct parcel) = 44 bytes (4 id + 4 weight + 24 address + 12 date). A swap uses three structure assignments (tmp = a; a = b; b = tmp), so each swap copies 3 x 44 = **132 bytes**. This data needs 2 swaps, so 2 x 132 = 264 bytes are copied. In the worst case (n-1 swaps) the sort copies 132(n-1) bytes.

Alternative Concept and Complexity Analysis: Flat Structure

Flat version (address and date fields written directly inside the parcel)

```
struct parcel_flat {
   int   id;
   float weight;
   char  city[20];  int pincode;        /* receiver */
   int   day, month, year;             /* booking date */
};
```

	Nested structures	**Flat structure**
Access path (receiver PIN, parcel pointer p)	p->to.pincode (or p->receiver.addr.pincode)	p->pincode: shorter, but names must encode the role
Reuse of the address type	One struct address serves sender and receiver, and a function such as print_address(const struct address *) works on both	Fields must be repeated (s_city, s_pin, r_city, r_pin); no address-level function can be shared; a change to the city length must be made in several places
Passing a part	Can pass &p->to alone	Cannot pass "the address" as one unit
Run-time cost	Same: member offsets are fixed at compile time, so the machine code is the same	Same

	Nested structures	Flat structure
Space (measured)	Exercise parcel: 44 bytes. Full parcel with sender, receiver, name, service and charge: 140 bytes	Exercise parcel: 44 bytes. Full flat parcel: 140 bytes

The space is identical because every member needs 4-byte alignment and no padding is inserted in either layout. Nesting costs nothing in memory or speed and gains reuse, clarity and safer passing of sub-records.

Task 3: Unions and Memory Layout (9 Marks)

Task 3.1: Structure or Union? (3 Marks)

Design A: structure holding all three fields

```
struct packet_all {
    enum kind type;
    float celsius;
    int   percent;
    char  status[8];      /* up to 7 characters + '\0' */
};
```

Design B: tagged union

```
enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;              /* tag: tells which member is valid */
    union {
        float celsius;
        int   percent;
        char  status[8];
    } value;
};
```

Justification. A packet carries exactly one kind of value, so the three fields are never needed at the same time. Design A reserves space for all three in every packet, so two of the three fields are always wasted. Design B overlays them, so each packet only needs room for the largest member plus a tag. With thousands of packets and very little memory, Design B is the right choice. At 10,000 packets it saves 80,000 bytes (about 78 KB, a 40% saving). Its cost is that the programmer must always check the tag before reading a member, which is why the union is never used without the tag.

Task 3.2: Predict the Size (2 Marks)

Design	Layout (typical 64-bit GCC)	Predicted / measured
Union alone	Members: float 4, int 4, char[8] 8. All start at offset 0 and overlap. Size = largest member = 8. Alignment = 4 (strictest member is float/int).	8 / 8
Design A (all fields)	type 0-3, celsius 4-7, percent 8-11, status 12-19. Total 20, which is already a multiple of the alignment 4, so no padding.	20 / 20
Design B (tagged union)	type 0-3, union starts at offset 4 (a multiple of its alignment 4), union 4-11. Total 12, multiple of 4, so no padding.	12 / 12

Alignment and padding. Each member must start at an address that is a multiple of its alignment, and the structure size must be a multiple of the strictest member alignment. Here every member needs at most 4-byte alignment, so no padding bytes are inserted. If the union contained a double (alignment 8), the

compiler would insert 4 padding bytes after the tag and the structure would grow to 16 bytes. The values 8, 20 and 12 were confirmed with sizeof.

Task 3.3: Implement and Test (3 Marks)

The complete program, including show() (the answer to TODO 1) and the misuse experiment of Task 3.4:

```
#include <stdio.h>
#include <string.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
    union { float celsius; int percent; char status[8]; } value;
};

struct reading_all {            /* "all three fields" design */
    enum kind type;
    float celsius; int percent; char status[8];
};

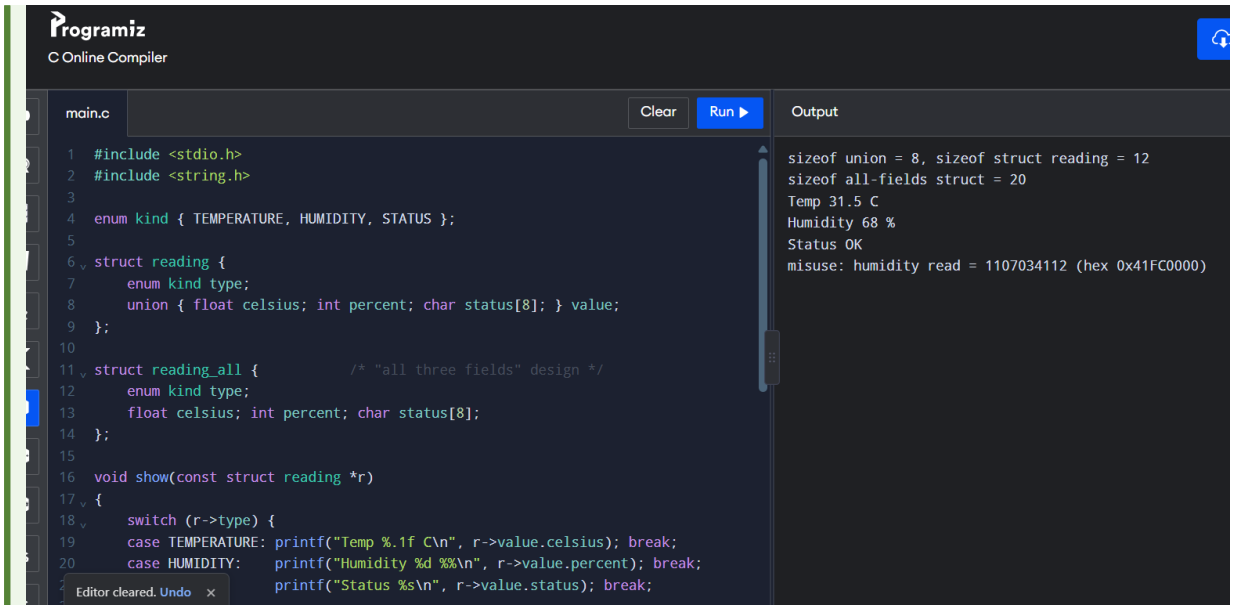
void show(const struct reading *r)
{
    switch (r->type) {
    case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
    case HUMIDITY:    printf("Humidity %d %%\n", r->value.percent); break;
    case STATUS:      printf("Status %s\n", r->value.status); break;
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    printf("sizeof all-fields struct = %zu\n", sizeof(struct reading_all));
    for (int i = 0; i < 3; i++) show(&log[i]);

    /* misuse experiment */
    struct reading m; m.type = TEMPERATURE; m.value.celsius = 31.5f;
    printf("misuse: humidity read = %d (hex 0x%X)\n", m.value.percent, (unsigned)m.value.percent);
    return 0;
}
```

Output

```
sizeof union = 8, sizeof struct reading = 12
sizeof all-fields struct = 20
Temp 31.5 C
Humidity 68 %
Status OK
misuse: humidity read = 1107034112 (hex 0x41FC0000)
```



Why 12 bytes and not 4 + 4 + 4 + 8? The sum 4 + 4 + 4 + 8 = 20 would be the size if the three values were separate fields (that is Design A). In a union, all members share the same memory starting at one address, so the union occupies only the size of its largest member (8 bytes), not 4 + 4 + 8. The structure is then the 4-byte tag plus the 8-byte union = 12 bytes.

Task 3.4: Misuse Experiment (1 Mark)

A temperature of 31.5 is stored in value.celsius, then value.percent is read. The output is **1107034112** (hexadecimal 0x41FC0000).

Explanation. celsius and percent share the same 4 bytes. 31.5 is stored in IEEE-754 single-precision format: sign 0, exponent 10000011 (that is 127 + 4), fraction 11111000...0, which makes the bit pattern 0x41FC0000. Reading the same bits as an int does no conversion; it merely reinterprets the pattern, giving 1107034112. The number is meaningless as a humidity. This is the reason for the type tag: the program must read only the member that was last written.

Alternative Concept and Complexity Analysis

Alternative 1: three separate arrays (float temp[N], int hum[N], char status[N][8]). **Alternative 2: each packet allocated with malloc** and referenced through a void * (a tag is still needed, because a void * carries no type).

	Tagged union array	Three separate arrays	malloc + void *
Time to read packet i	O(1): one indexed load, contiguous memory	O(1) if the type and index are known; the arrival order across the three types is lost unless an extra (type, index) table is kept	O(1) but needs an extra pointer dereference, a likely cache miss, and a cast; storing needs malloc (slower, not constant-time guaranteed)
Space per packet	12 bytes	4 / 4 / 8 bytes with no tag, but each array must be sized for the worst case, so unused capacity is wasted; plus the (type, index) table (about 8 bytes per packet) to keep order	Pointer 8 + tag 4 (+4 padding) = 16 bytes in the array, plus the heap block (typically 24 bytes minimum on glibc, with 8 to 16 bytes of malloc bookkeeping): about 40 bytes

	Tagged union array	Three separate arrays	malloc + void *
Other issues	Compact; the tag must be respected	Awkward to process packets in time order	Memory leaks, fragmentation, type errors from wrong casts, free needed for every packet

For a station with very little memory that stores thousands of packets, the tagged union is the best design: smallest per packet, constant-time access and no allocation.

Task 4: Structure Forensics (7 Marks)

Task 4.1: Locate the Defects (2 Marks)

- **Balances unchanged (logic defect).** addInterest(struct account a, ...) takes the structure **by value**. C copies list[i] into the parameter a; the function adds interest to this private copy, and the copy is destroyed when the function returns. The original list[i] is never touched.
- **Statement that does not compile.** In printf("%d %f\n", p.number, p.balance);, p is a **pointer** (struct account *p = list;). The dot operator needs a structure, not a pointer, so the compiler reports "'p' is a pointer; did you mean to use '->'?". The correct forms are p->number or (*p).number.
- Additional points: the program lacks #include <stdio.h> (so printf is undeclared), and p points only to list[0], so even after the fix only the first account would be printed.

Output

```
t4_orig.c:11:24: error: 'p' is a pointer; did you mean to use '->'?
t4_orig.c:11:34: error: 'p' is a pointer; did you mean to use '->'?
```

Task 4.2: Construct Evidence (2 Marks)

Printing the address of the structure in both functions shows that they are different objects:

```
#include <stdio.h>

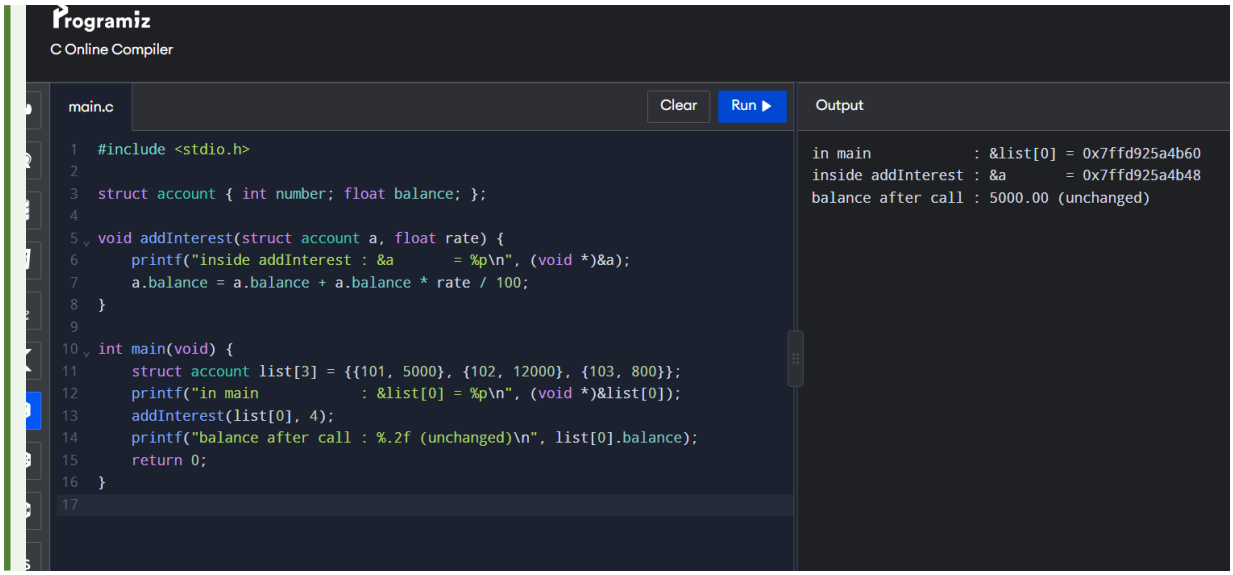
struct account { int number; float balance; };

void addInterest(struct account a, float rate) {
    printf("inside addInterest : &a       = %p\n", (void *)&a);
    a.balance = a.balance + a.balance * rate / 100;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    printf("in main            : &list[0] = %p\n", (void *)&list[0]);
    addInterest(list[0], 4);
    printf("balance after call : %.2f (unchanged)\n", list[0].balance);
    return 0;
}
```

Output

```
in main            : &list[0] = 0x7ffd3cd46e10
inside addInterest : &a       = 0x7ffd3cd46df8
balance after call : 5000.00 (unchanged)
```



The two addresses are different, so `a` is a separate copy (on the called function's stack frame), and the balance in `main` is still 5000.00. Absolute addresses vary between runs; what matters is that they are not equal.

Task 4.3: Repair the Program (2 Marks)

```
#include <stdio.h>

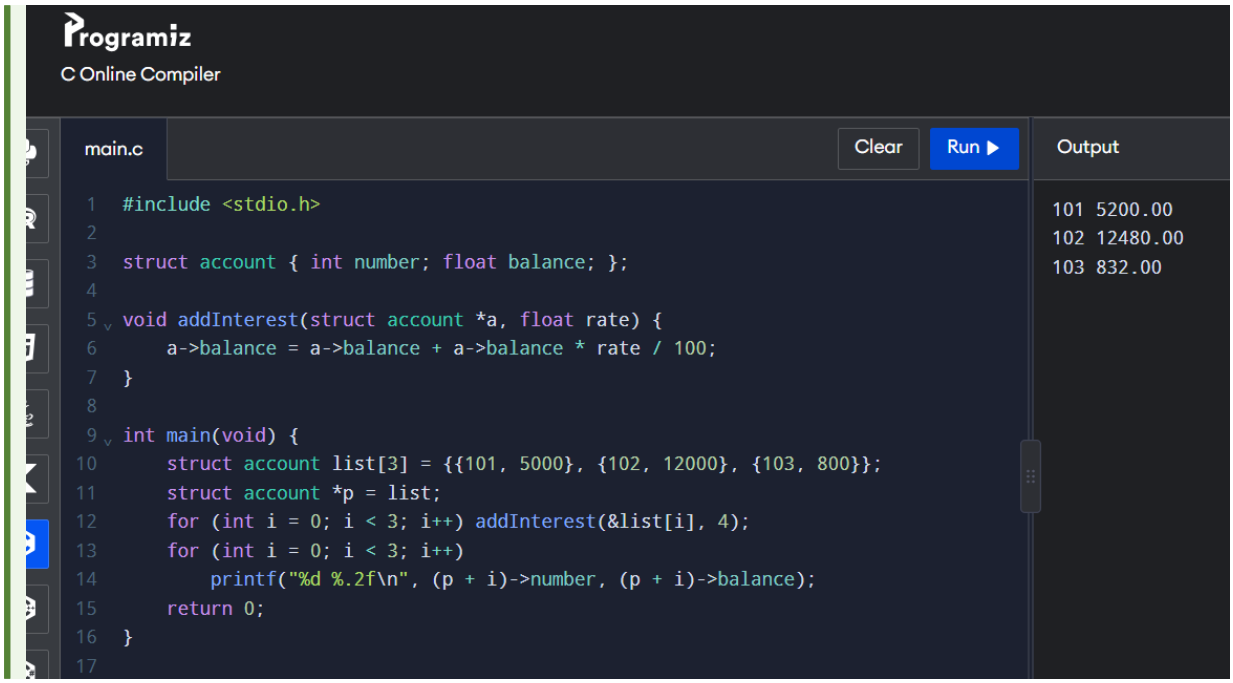
struct account { int number; float balance; };

void addInterest(struct account *a, float rate) {
    a->balance = a->balance + a->balance * rate / 100;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;
    for (int i = 0; i < 3; i++) addInterest(&list[i], 4);
    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", (p + i)->number, (p + i)->balance);
    return 0;
}
```

Output

```
101 5200.00
102 12480.00
103 832.00
```



The function receives the address of the real structure (&list[i]) and the arrow operator a->balance modifies the original. The print statement uses (p + i)->number so all three accounts are shown.

Task 4.4: Prevent Recurrence (1 Mark)

Pass	Use when	Example
By value	The structure is small and the function needs its own private copy, or must not affect the caller.	double area(struct rect r)
By pointer	The function must modify the caller's structure, or the structure is large and copying is wasteful.	void addInterest(struct account *a, float rate)
By const pointer	The function only reads a (usually large) structure. This avoids the copy and the compiler rejects accidental writes.	void print_parcel(const struct parcel *p)

Alternative Concept and Complexity Analysis: Return by Value

```
struct account addInterest(struct account a, float rate) {
    a.balance = a.balance + a.balance * rate / 100;
    return a;                    /* copy of the modified structure */
}
/* caller */
list[i] = addInterest(list[i], 4);
```

This works because the modified copy is returned and assigned back over the original. With sizeof(struct account) = 8 bytes (measured):

	Pass by value and return the copy	Pass a pointer
Bytes copied per call	8 (argument) + 8 (return value) = 16, plus 8 more for the assignment into list[i] if the compiler does not elide it	8 (one pointer)
Time per call	O(S) for a structure of S bytes; O(1) here because S is fixed and small. For n accounts, O(n) in total	O(1) per call, independent of S; O(n) in total

	Pass by value and return the copy	Pass a pointer
Stack space	About 2S for the parameter and the returned copy (16 bytes here), plus the frame	One pointer (8 bytes), plus the frame
Risk	Forgetting the assignment back (addInterest(list[i], 4); alone) repeats the original bug	None for this bug; the original is modified directly

For an 8-byte structure the difference is negligible (some ABIs even pass it in registers). For a 1 KB structure the by-value version would copy about 2 KB per call, so the pointer version scales better, and it also states the intent "modify in place" more clearly.

Task 5: Storage Classes: The Same Counter Four Ways (7 Marks)

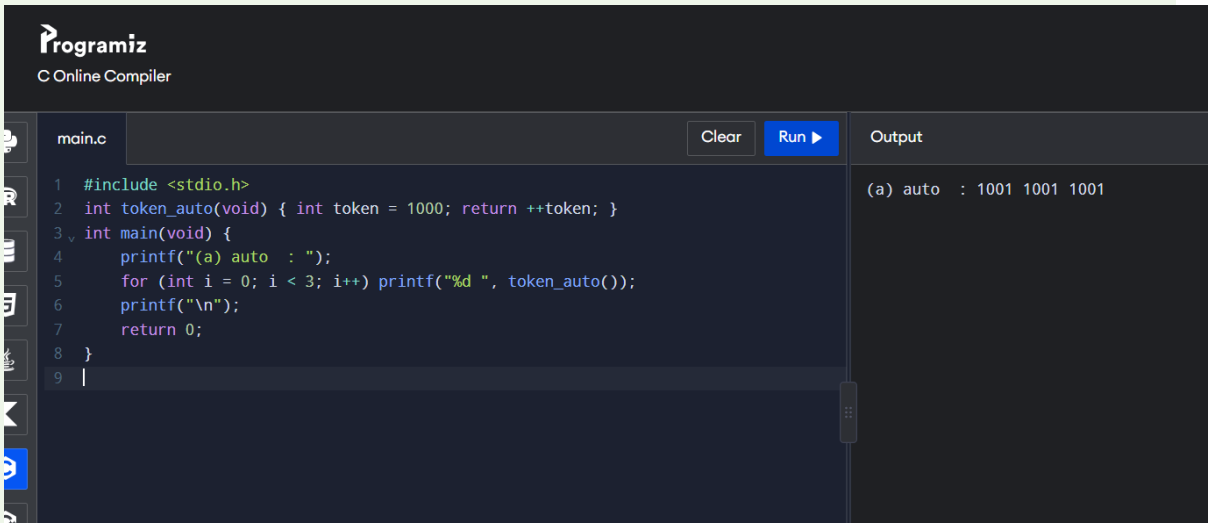
Task 5.1: Four Implementations (3 Marks)

(a) Automatic local variable

```
#include <stdio.h>
int token_auto(void) { int token = 1000; return ++token; }
int main(void) {
    printf("(a) auto  : ");
    for (int i = 0; i < 3; i++) printf("%d ", token_auto());
    printf("\n");
    return 0;
}
```

Output

```
(a) auto  : 1001 1001 1001
```



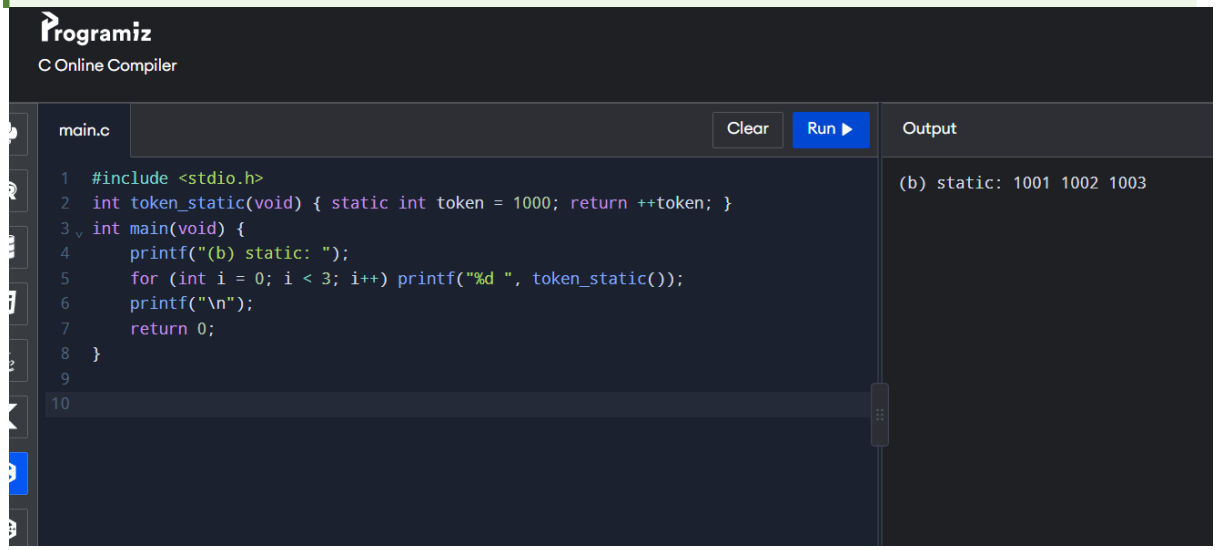
The variable is created and re-initialised to 1000 on every call, so every call returns 1001. An automatic variable cannot remember anything between calls.

(b) Static local variable

```
#include <stdio.h>
int token_static(void) { static int token = 1000; return ++token; }
int main(void) {
    printf("(b) static: ");
    for (int i = 0; i < 3; i++) printf("%d ", token_static());
    printf("\n");
    return 0;
}
```

Output

```
(b) static: 1001 1002 1003
```



The static variable is initialised once (before the program starts) and keeps its value for the whole run, while still being visible only inside the function.

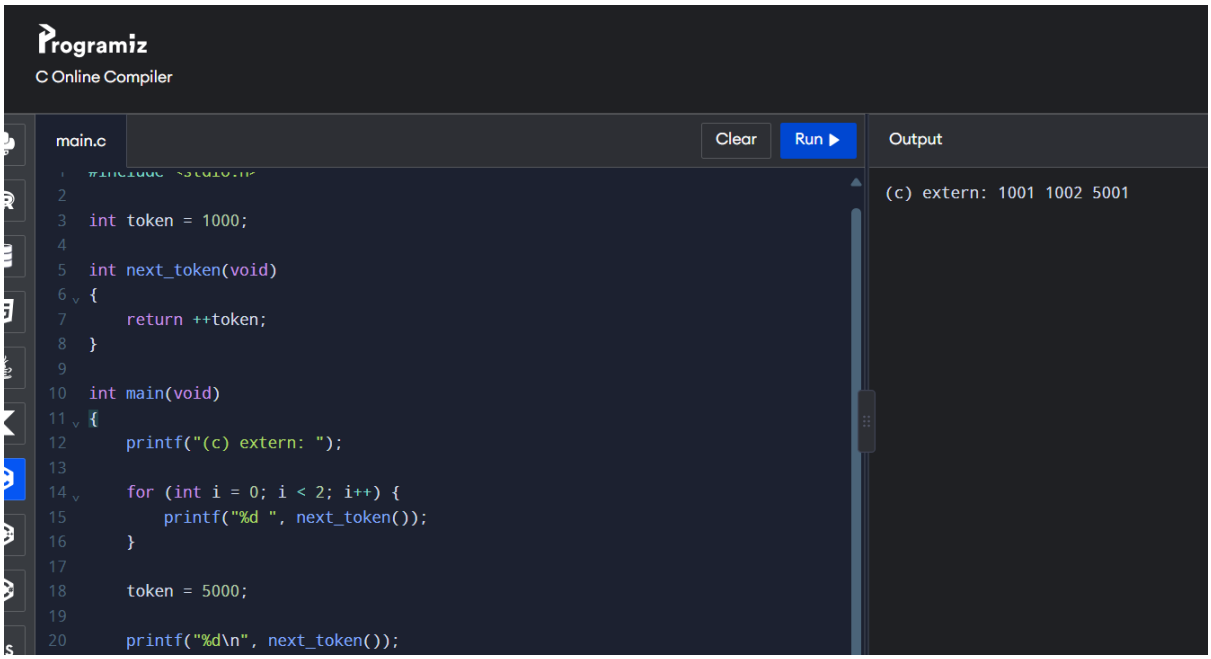
(c) Global variable shared across two files with extern

```
/* file 1: defines the shared global */
int token = 1000;
int next_token(void) { return ++token; }
```

```
/* file 2: uses the same global through extern */
#include <stdio.h>
extern int token;
extern int next_token(void);
int main(void) {
    printf("(c) extern: ");
    for (int i = 0; i < 2; i++) printf("%d ", next_token());
    token = 5000;               /* any file can corrupt it! */
    printf("%d\n", next_token());
    return 0;
}
```

Output

```
(c) extern: 1001 1002 5001
```



File 1 defines token; file 2 refers to the same variable through extern. The counter works across files, but file 2 was also able to set token = 5000, so the next token became 5001. This shows the accidental-modification risk. (Build: gcc c_counter.c c_main.c.)

(d) Register loop variable in the printing loop

```
#include <stdio.h>
static int token = 1000;
int next_token(void) { return ++token; }
int main(void) {
    printf("(d) register loop: ");
    register int i;
    for (i = 0; i < 3; i++) printf("%d ", next_token());
    printf("\n");
    /* int *p = &i;   <-- error: address of register variable 'i' requested */
    return 0;
}
```

Output

```
(d) register loop: 1001 1002 1003
```



The loop counter i is a register variable (a request to keep it in a CPU register for speed). It controls the loop only; the token numbers still come from a counter that lives elsewhere (here, a file-scope static). A register variable cannot itself carry the counter between calls.

Version	Tokens recorded	Correct?
(a) automatic	1001 1001 1001	No (state is lost)
(b) static local	1001 1002 1003	Yes
(c) global with extern	1001 1002, then 5001 after outside tampering	Yes, but not protected
(d) register loop variable	1001 1002 1003	Yes (counter is separate from the register variable)

Task 5.2: Change the Requirement (2 Marks)

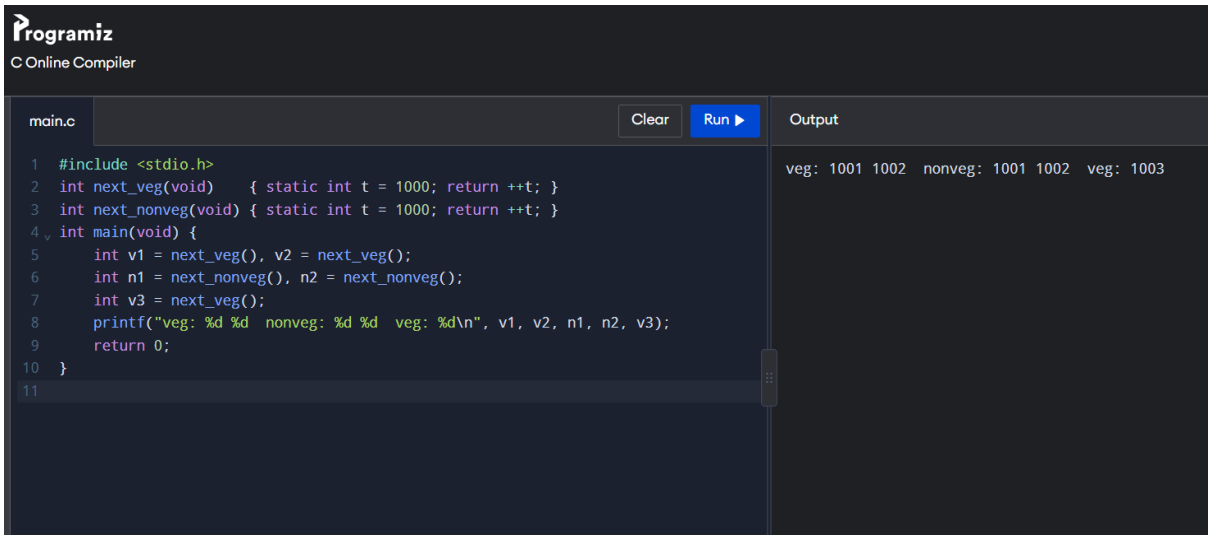
Two independent counters (veg and non-veg), each starting at 1001.

Static version

```
#include <stdio.h>
int next_veg(void)    { static int t = 1000; return ++t; }
int next_nonveg(void) { static int t = 1000; return ++t; }
int main(void) {
    int v1 = next_veg(), v2 = next_veg();
    int n1 = next_nonveg(), n2 = next_nonveg();
    int v3 = next_veg();
    printf("veg: %d %d  nonveg: %d %d  veg: %d\n", v1, v2, n1, n2, v3);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002  veg: 1003
```



Global version

```
#include <stdio.h>
int veg_token = 1000, nonveg_token = 1000;      /* two globals */
int next_veg(void)    { return ++veg_token; }
int next_nonveg(void) { return ++nonveg_token; }
int main(void) {
    int v1 = next_veg(), v2 = next_veg();
    int n1 = next_nonveg(), n2 = next_nonveg();
    printf("veg: %d %d  nonveg: %d %d\n", v1, v2, n1, n2);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002
```



Version	What had to change	Approximate code change
(a) automatic	Cannot meet the requirement at all: neither counter can remember anything. The storage class itself must change (to static, global or pointer-based) first.	Full redesign
(b) static local	A second function with its own static variable (next_veg, next_nonveg); all call sites change to choose the right function.	+1 function (3 lines); call sites edited
(c) global / extern	A second global, a second function, and an extern declaration for each new name in every file that uses it.	+1 global, +1 function, +1 extern per file (about 5 lines or more)
(d) register loop variable	The loop variable is unaffected; the change happens in whichever counter version it prints from.	0 lines in the loop itself

In (b) and (c) the **function is duplicated** for every new counter. With 10 counters there would be 10 near-identical functions, which does not scale.

Task 5.3: Compare the Storage Classes (2 Marks)

	auto	static local	global (extern)	register
Scope	Block (function)	Block (function)	File scope; visible in other files through extern	Block
Lifetime	While the block runs	Whole program	Whole program	While the block runs
Initial value	Indeterminate (garbage) unless initialised	0 by default; initialised once, before main	0 by default; initialised once	Indeterminate unless initialised
Accidental-modification risk	Low (private) but loses state	Low (private to the function), but hidden state and not thread-safe	High: any function in any file can change it; name clashes	Low (private)

Why the address of a register variable cannot be taken. register asks the compiler to keep the variable in a CPU register. Registers do not have memory addresses, so the address-of operator & has nothing to return. The C standard therefore makes & on a register variable a constraint violation. GCC reports it:

Output

```
bad.c:1:1: error: address of register variable 'i' requested
```

Modern compilers ignore register as a speed hint, because they allocate registers better themselves, but the language rule still applies.

Complete the Code (TODO 1)

```c
#include <stdio.h>

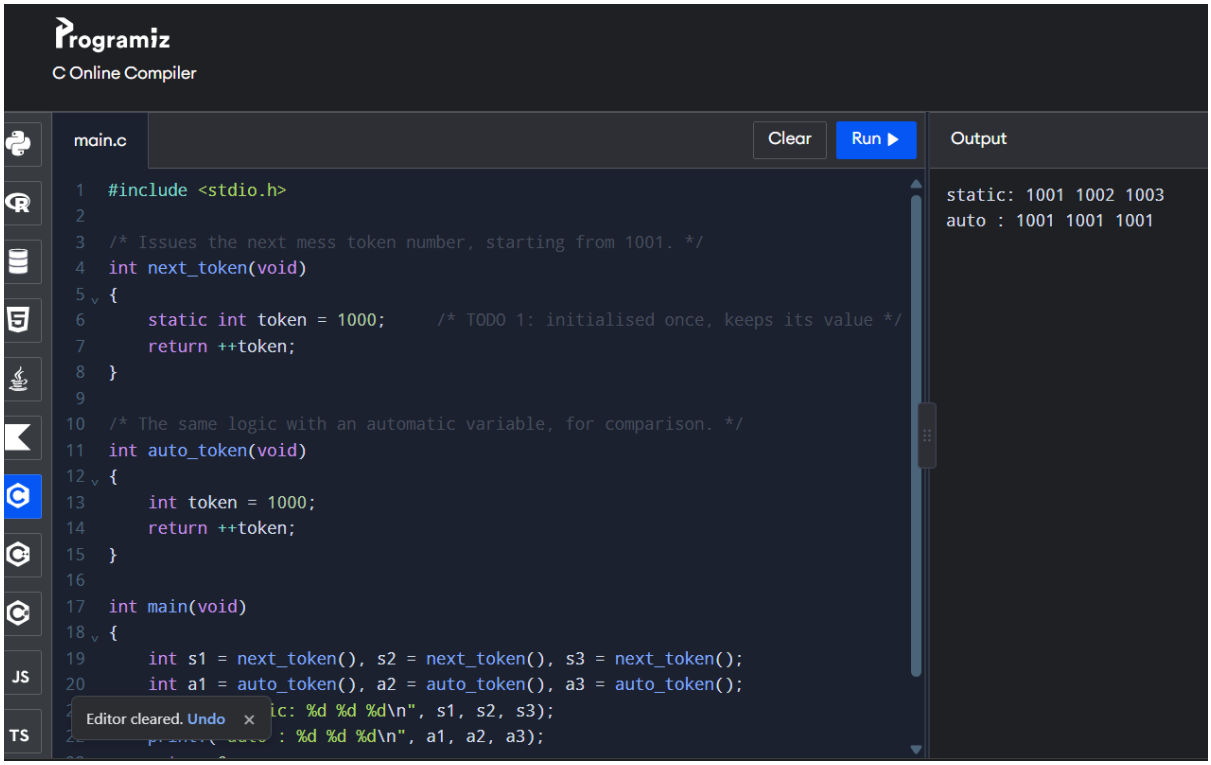
/* Issues the next mess token number, starting from 1001. */
int next_token(void)
{
    static int token = 1000;     /* TODO 1: initialised once, keeps its value */
    return ++token;
}

/* The same logic with an automatic variable, for comparison. */
int auto_token(void)
{
    int token = 1000;
    return ++token;
}

int main(void)
{
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);
    return 0;
}
```

Output

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

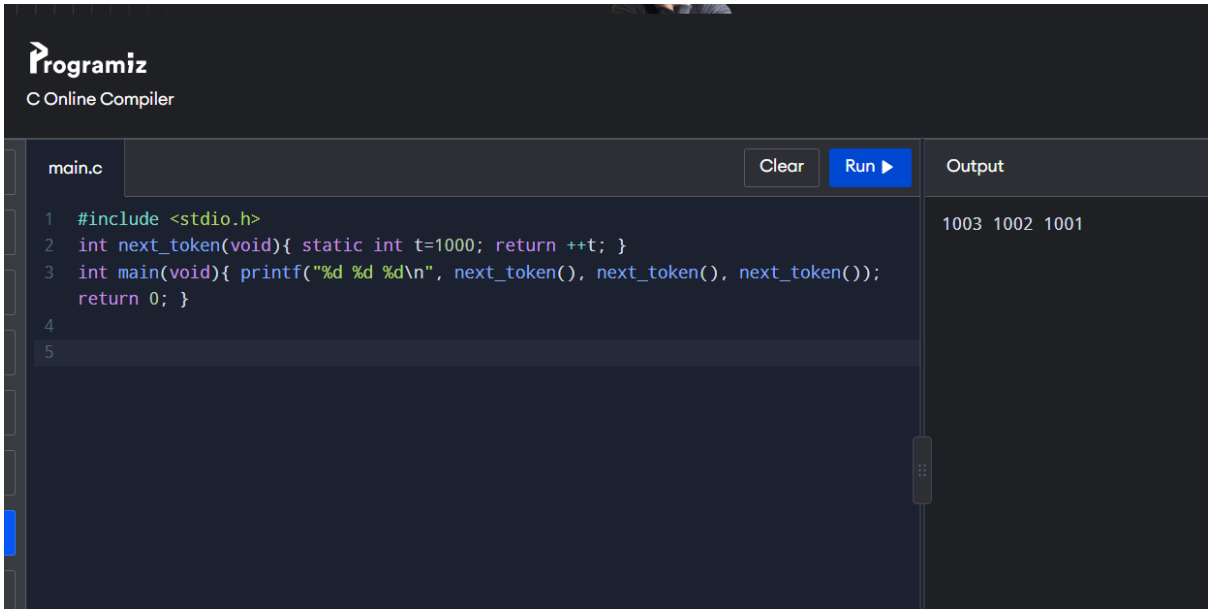


Order-of-evaluation experiment

```
#include <stdio.h>
int next_token(void){ static int t=1000; return ++t; }
int main(void){ printf("%d %d %d\n", next_token(), next_token(), next_token()); return 0; }
```

Output

```
1003 1002 1001
```



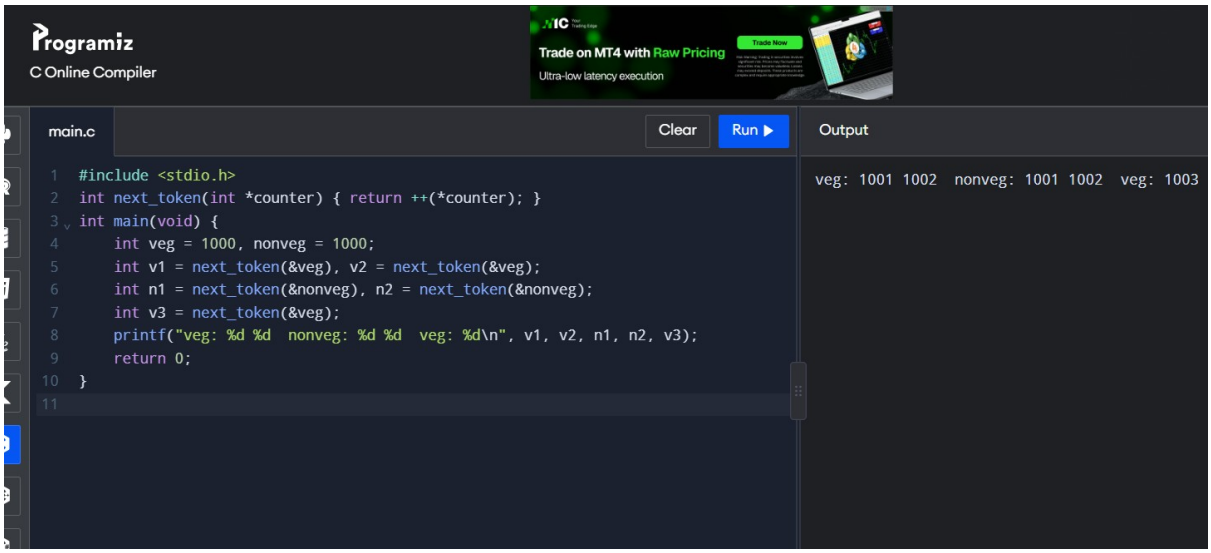
The three calls in one printf are the function arguments. The C standard does **not specify** the order in which function arguments are evaluated, so the compiler may call next_token() in any order. GCC on x86-64 usually evaluates the arguments from right to left, so the rightmost call gets 1001 and the leftmost gets 1003. Another compiler (or another optimisation level) may print 1001 1002 1003. The program is not undefined, but its output is unspecified and must not be relied on. The fix is to make separate statements (int a = next_token(); int b = next_token(); ...) as in the main program.

Alternative Concept and Complexity Analysis: Counter Owned by the Caller

```
#include <stdio.h>
int next_token(int *counter) { return ++(*counter); }
int main(void) {
    int veg = 1000, nonveg = 1000;
    int v1 = next_token(&veg), v2 = next_token(&veg);
    int n1 = next_token(&nonveg), n2 = next_token(&nonveg);
    int v3 = next_token(&veg);
    printf("veg: %d %d  nonveg: %d %d  veg: %d\n", v1, v2, n1, n2, v3);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002  veg: 1003
```



Aspect	Result
Time	O(1) per call in all versions: one increment (the pointer version adds one dereference).
Space	One int (4 bytes) per counter, plus an 8-byte pointer argument on the stack during the call. No hidden storage in the function.
Lifetime	Decided by the caller: the counter lives as long as the caller's variable (a local in main here), not for the whole program. It can be reset, saved or destroyed explicitly.
Two independent counters	Trivial: declare another int and pass its address. One line of change and the function body stays untouched, whereas the static and global versions need a new function each.
Safety	Only code that holds the pointer can change the counter, so there is no global exposure; the function is re-entrant and thread-friendly. The caller must pass a valid pointer.

Evaluation. The pointer version is the best of all: it scales to any number of counters without duplication, and it avoids the hidden state of static and the open access of global variables.

Task 6: Design Challenge: College Canteen Inventory and Billing (13 Marks)

Stage 1: Interpret the Problem (2 Marks)

Entity	Data
Item (common)	id, name, kind (packaged, cooked or combo), price, stock, units sold, revenue
Packaged item	common data + expiry date
Cooked item	common data + time prepared (hour, minute)
Combo meal	price + up to 3 component item ids (stock is taken from its components)
Date / Time	day, month, year / hour, minute
Sale	item id, quantity (recorded into the item's sold count and revenue)
Canteen	array of items, number of items, today's date

Requirements as testable statements:

ID	Requirement	Test
R1	A sale of quantity q is accepted only if q > 0 and the item exists.	Sell 0 units or id 99: rejected
R2	A sale is rejected if stock < q, and the stock is left unchanged.	Samosa stock 4, sell 10: rejected, stock still 4
R3	After an accepted sale, stock falls by exactly q and revenue rises by q x price.	Sell 5 biscuits at 10: stock 20 to 15, revenue 50
R4	A combo sale needs every component to have stock >= q; either all components are reduced or none are.	3 combos reduce dosa by 3 and juice by 3
R5	A packaged item that expires within 2 days (0 to 2 days from today) triggers a warning; an expired one is flagged and cannot be sold.	Juice expires tomorrow: warning
R6	The summary lists items in descending order of revenue, and the total equals the sum of item revenues.	Order 180, 160, 50, 30, 0; total 420
R7	A combo has 1 to 3 components.	Combo with 4 components not accepted

Stage 2: Propose the Data Model (2 Marks)

Mechanism	Where it is used and why
Nested structures	struct date (expiry) and struct time (time prepared) are nested in the item; struct combo_info (component count and ids) is nested in the union.
Tagged union	An item is exactly one of three kinds. The common fields (id, name, price, stock, sold, revenue) stay outside; the kind-specific data (expiry, prepared time, combo components) share one union, selected by the tag enum item_kind kind.
Array of structures	struct item items[MAX_ITEMS] inside struct canteen, together with the item count and today's date. Items are stored contiguously for easy linear search.
Array of pointers	For the revenue-sorted summary, an array of struct item * is sorted instead of moving whole 64-byte items.

Stage 3: Identify the Critical Design Decisions (2 Marks)

- **Safe stock update:** always check quantity and stock first and change stock only afterwards. For combos use two passes: pass 1 validates every component without changing anything; pass 2 deducts. This guarantees no partial deduction, so stock can never become negative. The function modifies the item in place through a pointer (struct item *), not through a copy, and returns a status code (enum sale_status) so that the caller knows why a sale failed.
- **Expiry check with a date structure:** converting a date to a day number (days since a fixed origin) and subtracting gives the exact number of days left, with no mistakes at month ends or leap years. days_until(today, expiry) < 0 means expired; 0 to 2 means warn. Comparing day, month and year fields one by one does not give "within two days".
- **Static or global data:** the canteen object is large and must start empty, so it is a static object (zero-initialised), created in one place and passed by pointer everywhere. The helper functions day_number and days_until are static (file-private). The status-message table is a static const array. MAX_ITEMS and WARN_DAYS are constants (#define). Today's date is kept in the canteen structure rather than a mutable global, so it can be changed for testing. No other global variable is used.

Stage 4: Design the Structures and Functions (4 Marks)

Structure definitions

```
#define MAX_ITEMS 20
```

```
#define WARN_DAYS 2

struct date { int day, month, year; };
struct time { int hour, minute; };

enum item_kind { PACKAGED, COOKED, COMBO };
enum sale_status { SALE_OK, SALE_NO_ITEM, SALE_BAD_QTY, SALE_NO_STOCK, SALE_EXPIRED };

struct combo_info { int count; int item_id[3]; };

struct item {
    int   id;
    char  name[24];
    enum item_kind kind;                /* tag */
    float price;
    int   stock;                        /* unused for COMBO */
    int   sold;
    float revenue;
    union {
        struct date       expiry;       /* PACKAGED */
        struct time       prepared;     /* COOKED   */
        struct combo_info combo;        /* COMBO    */
    } u;
};

struct canteen {
    struct item items[MAX_ITEMS];
    int count;
    struct date today;
};
```

Function prototypes (all use structure pointers)

```
struct item     *find_item(struct canteen *c, int id);
int             add_item(struct canteen *c, const struct item *it);
enum sale_status record_sale(struct canteen *c, int id, int qty);
void            expiry_warning(const struct canteen *c);
void            bill_summary(struct canteen *c);
static long     day_number(const struct date *d);
static int      days_until(const struct date *from, const struct date *to);
```

Pseudocode: record a sale

```
RECORD_SALE(c, id, qty)
  it <- FIND_ITEM(c, id)
  if it = NULL         then return NO_ITEM
  if qty <= 0          then return BAD_QTY
  if it.kind = COMBO then
     for each component k of it                  // pass 1: validate only
        part <- FIND_ITEM(c, k)
        if part = NULL or part.stock < qty then return NO_STOCK
        if part is PACKAGED and DAYS_UNTIL(today, part.expiry) < 0
                                          then return EXPIRED
     for each component k of it                  // pass 2: commit
        FIND_ITEM(c, k).stock <- stock - qty
  else
     if it.stock < qty                    then return NO_STOCK
     if it is PACKAGED and DAYS_UNTIL(today, it.expiry) < 0
                                       then return EXPIRED
     it.stock <- it.stock - qty
  it.sold    <- it.sold + qty
  it.revenue <- it.revenue + qty * it.price
  return OK
```

Pseudocode: expiry warning

```
EXPIRY_WARNING(c)
  any <- false
  for each item it in c.items
     if it.kind != PACKAGED then continue
     left <- DAYS_UNTIL(c.today, it.expiry)       // via day numbers
     if left < 0           then print "EXPIRED", it.name ; any <- true
     else if left <= 2     then print "WARNING", it.name, left ; any <- true
  if not any then print "No items expiring within 2 days"
```

Stage 5: Implement and Demonstrate (3 Marks)

Full program (compiled and run). It demonstrates normal sales, a sale that exceeds stock, an expiring item and the sorted summary. Today is fixed at 12/11/2026.

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 20
#define WARN_DAYS 2

struct date { int day, month, year; };
struct time { int hour, minute; };

enum item_kind { PACKAGED, COOKED, COMBO };
enum sale_status { SALE_OK, SALE_NO_ITEM, SALE_BAD_QTY, SALE_NO_STOCK, SALE_EXPIRED };

struct combo_info { int count; int item_id[3]; };

struct item {
    int   id;
    char  name[24];
    enum item_kind kind;                /* tag */
    float price;
    int   stock;                        /* unused for COMBO */
    int   sold;
    float revenue;
    union {
        struct date       expiry;       /* PACKAGED */
        struct time       prepared;     /* COOKED   */
        struct combo_info combo;        /* COMBO    */
    } u;
};

struct canteen {
    struct item items[MAX_ITEMS];
    int count;
    struct date today;
};

/* ---------- date helper: days since 1970-01-01 ---------- */
static long day_number(const struct date *d)
{
    int y = d->year, m = d->month;
    if (m <= 2) { y--; m += 12; }
    return 365L * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d->day;
}

static int days_until(const struct date *from, const struct date *to)
{
    return (int)(day_number(to) - day_number(from));
}

/* ---------- operations ---------- */
struct item *find_item(struct canteen *c, int id)
{
    for (int i = 0; i < c->count; i++)
        if (c->items[i].id == id) return &c->items[i];
    return NULL;
}

int add_item(struct canteen *c, const struct item *it)
{
    if (c->count >= MAX_ITEMS) return 0;
    c->items[c->count++] = *it;
    return 1;
}

enum sale_status record_sale(struct canteen *c, int id, int qty)
{
```

```
    struct item *it = find_item(c, id);
    if (it == NULL)  return SALE_NO_ITEM;
    if (qty <= 0)    return SALE_BAD_QTY;

    if (it->kind == COMBO) {
        /* PASS 1: validate every component, change nothing */
        for (int k = 0; k < it->u.combo.count; k++) {
            struct item *part = find_item(c, it->u.combo.item_id[k]);
            if (part == NULL || part->stock < qty) return SALE_NO_STOCK;
            if (part->kind == PACKAGED && days_until(&c->today, &part->u.expiry) < 0)
                return SALE_EXPIRED;
        }
        /* PASS 2: commit */
        for (int k = 0; k < it->u.combo.count; k++)
            find_item(c, it->u.combo.item_id[k])->stock -= qty;
    } else {
        if (it->stock < qty) return SALE_NO_STOCK;
        if (it->kind == PACKAGED && days_until(&c->today, &it->u.expiry) < 0)
            return SALE_EXPIRED;
        it->stock -= qty;
    }
    it->sold    += qty;
    it->revenue += qty * it->price;
    return SALE_OK;
}

void expiry_warning(const struct canteen *c)
{
    int any = 0;
    for (int i = 0; i < c->count; i++) {
        const struct item *it = &c->items[i];
        if (it->kind != PACKAGED) continue;
        int left = days_until(&c->today, &it->u.expiry);
        if (left < 0)
            { printf("  EXPIRED : %-16s (expired %d day(s) ago)\n", it->name, -left); any = 1; }
        else if (left <= WARN_DAYS)
            { printf("  WARNING : %-16s expires in %d day(s)\n", it->name, left); any = 1; }
    }
    if (!any) printf("  No items expiring within %d days.\n", WARN_DAYS);
}

void bill_summary(struct canteen *c)
{
    struct item *order[MAX_ITEMS];          /* sort pointers, not structures */
    for (int i = 0; i < c->count; i++) order[i] = &c->items[i];

    for (int i = 0; i < c->count - 1; i++) {   /* selection sort, descending */
        int max = i;
        for (int j = i + 1; j < c->count; j++)
            if (order[j]->revenue > order[max]->revenue) max = j;
        struct item *t = order[i]; order[i] = order[max]; order[max] = t;
    }
    float total = 0;
    printf("  %-16s %5s %9s\n", "Item", "Sold", "Revenue");
    for (int i = 0; i < c->count; i++) {
        printf("  %-16s %5d %9.2f\n", order[i]->name, order[i]->sold, order[i]->revenue);
        total += order[i]->revenue;
    }
    printf("  %-16s %5s %9.2f\n", "TOTAL", "", total);
}

static void report(const char *what, enum sale_status s)
{
    static const char *msg[] = {"OK", "item not found", "bad quantity",
                                "REJECTED - insufficient stock", "REJECTED - item expired"};
    printf("  %-26s -> %s\n", what, msg[s]);
}

int main(void)
{
    static struct canteen c = { .count = 0, .today = {12, 11, 2026} };

    struct item biscuit = {1, "Biscuit pack", PACKAGED, 10.0f, 20, 0, 0, .u.expiry = {30, 11, 2026}};
```

```
    struct item juice   = {2, "Juice box",    PACKAGED, 25.0f, 10, 0, 0, .u.expiry = {13, 11, 2026}};
    struct item dosa    = {3, "Masala dosa",  COOKED,   40.0f, 15, 0, 0, .u.prepared = {8, 30}};
    struct item samosa  = {4, "Samosa",       COOKED,   15.0f,  6, 0, 0, .u.prepared = {9, 15}};
    struct item combo   = {5, "Dosa+Juice",   COMBO,    60.0f,  0, 0, 0, .u.combo = {2, {3, 2}}};
    add_item(&c, &biscuit); add_item(&c, &juice); add_item(&c, &dosa);
    add_item(&c, &samosa);  add_item(&c, &combo);

    printf("TEST 1: normal sales\n");
    report("sell 5 biscuit packs",  record_sale(&c, 1, 5));
    report("sell 4 masala dosa",    record_sale(&c, 3, 4));
    report("sell 2 samosa",         record_sale(&c, 4, 2));
    report("sell 3 combos",         record_sale(&c, 5, 3));

    printf("\nTEST 2: sale exceeding stock\n");
    report("sell 10 samosa (stock 4)", record_sale(&c, 4, 10));
    printf("  samosa stock still = %d (unchanged)\n", find_item(&c, 4)->stock);

    printf("\nTEST 3: expiry warning (today = 12/11/2026)\n");
    expiry_warning(&c);

    printf("\nTEST 4: sorted bill summary (highest revenue first)\n");
    bill_summary(&c);
    return 0;
}
```

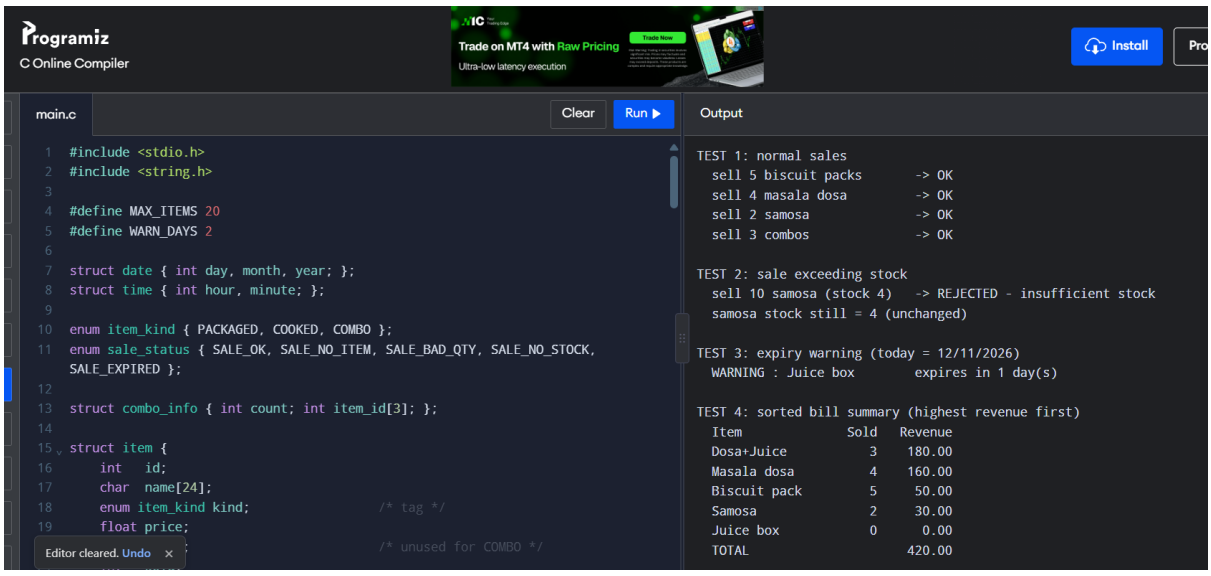
Output

```
TEST 1: normal sales
  sell 5 biscuit packs       -> OK
  sell 4 masala dosa         -> OK
  sell 2 samosa              -> OK
  sell 3 combos              -> OK

TEST 2: sale exceeding stock
  sell 10 samosa (stock 4)   -> REJECTED - insufficient stock
  samosa stock still = 4 (unchanged)

TEST 3: expiry warning (today = 12/11/2026)
  WARNING : Juice box        expires in 1 day(s)

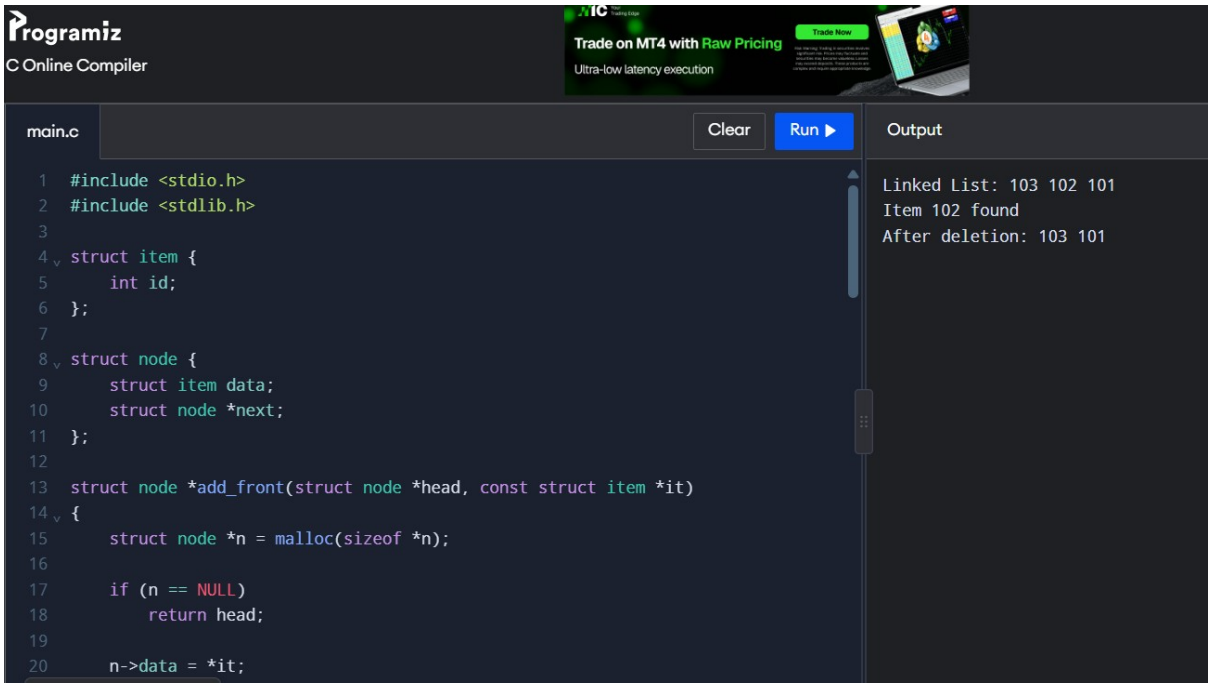
TEST 4: sorted bill summary (highest revenue first)
  Item              Sold   Revenue
  Dosa+Juice           3    180.00
  Masala dosa          4    160.00
  Biscuit pack         5     50.00
  Samosa               2     30.00
  Juice box            0      0.00
  TOTAL                     420.00
```



Alternative Concept and Complexity Analysis: Linked List

```
struct node {
    struct item  data;
    struct node *next;       /* extra pointer: 8 bytes */
};

struct node *add_front(struct node *head, const struct item *it) {
    struct node *n = malloc(sizeof *n);
    if (!n) return head;
    n->data = *it;  n->next = head;
    return n;                           /* O(1) */
}
struct node *find(struct node *h, int id) {      /* O(n) */
    for (; h; h = h->next) if (h->data.id == id) return h;
    return NULL;
}
struct node *remove_id(struct node *h, int id) { /* O(n) search + O(1) unlink */
    struct node *prev = NULL, *cur = h;
    while (cur && cur->data.id != id) { prev = cur; cur = cur->next; }
    if (!cur) return h;
    if (prev) prev->next = cur->next; else h = cur->next;
    free(cur);
    return h;
}
```



	Array of structures	Linked list of malloc'd structures
Add	O(1) at the end (if capacity remains); fails when MAX_ITEMS is reached	O(1) at the head (O(1) at the tail with a tail pointer; O(n) without). No capacity limit, but one malloc per item
Remove	O(n): find O(n), then shift up to n-1 elements of 64 bytes each	O(n) to find, then O(1) to unlink; no shifting; free the node
Search	O(n) linear; O(log n) with binary search if kept sorted by id	O(n); binary search is impossible
Extra space	None per item, but unused capacity is wasted: (MAX_ITEMS - n) x 64 bytes	One next pointer per item: 8n bytes, plus malloc bookkeeping (about 8 to 16 bytes per node)

	Array of structures	Linked list of malloc'd structures
Memory behaviour	Contiguous: excellent cache behaviour, no allocation failures after start	Scattered nodes: more cache misses; must check malloc failure and avoid leaks

Evaluation. For a canteen with a small, nearly fixed menu (about 20 items), the array is simpler and faster. A linked list is better only if the number of items changes frequently and has no sensible upper limit.

Overall Feedback and Conclusion

This assignment improved my understanding of structures, unions, pointers, storage classes and memory management in C. It helped me develop problem-solving, debugging and analytical skills. The practical programs and complexity analysis provided valuable experience in designing efficient and reliable data structures for real-world applications.